

Tập 6 - Flannel

Cài đặt Flannel & Giải mã Kiến trúc Định tuyến L2/L3 (VXLAN Mode)

Phần 1 — Flannel · `#flannel` `#CNI` `#overlay` `#flanneld` `#FDB-ARP`



Mục tiêu tập này

- Hiểu bài toán Pod-to-Pod cross-node communication và vai trò của CNI.
- Dựng cụm mạng Flannel từ con số 0 (từ cụm trạng NotReady).
- Khám phá kiến trúc 3 thành phần của Flannel: **K8s API**, **flanneld**, và **CNI plugin**.
- Phân tích chi tiết quy trình node join & giao tiếp 3 bước tĩnh của kernel: **Route** → **ARP** → **FDB**.

Bài toán: 2 Pod, 2 Node, không nói chuyện được

Node 1 (192.168.64.11)

Pod A: 10.244.1.5/24
Default route:
via 10.244.1.1 (cni0)

Node 2 (192.168.64.12)

Pod B: 10.244.2.7/24

Pod A gửi packet đến 10.244.2.7:
ip route show → không có route đến 10.244.2.0/24
→ packet bị DROP ngay tại Node 1 (no route to host)

Giải pháp Flannel: tạo virtual network phẳng (Overlay) qua VXLAN chui qua mạng vật lý.

3 thành phần cốt lõi của Flannel CNI

```
Kubernetes API / etcd
Node annotation: podCIDR + public-ip
controlplane: 10.244.0.0/24
worker1:      10.244.1.0/24
worker2:      10.244.2.0/24
```

watch

flanneld (cp) flanneld (w1) flanneld (w2) (DaemonSet)

- Lắng nghe sự kiện Node Added/Modified từ K8s API.
- Tạo VTEP interface (flannel.1), ghi /run/flannel/subnet.env.
- Cấu hình các bảng tĩnh ARP/FDB/Route trên Node Host.

CNI plugin (bridge binary)

Đọc subnet.env → gán IP cho Pod khi Kubelet tạo Pod.

Cơ cấu giao tiếp giữa flanneld và CNI plugin

- **flanneld** là "bộ não" quản lý Control Plane (watch API, setup routing table, static ARP, FDB).
- **CNI bridge plugin** là "tay chân" quản lý Data Plane ở local (tạo veth pair, gán IP Pod, cắm vào `cni0`).
- **subnet.env** là "hợp đồng" giao tiếp giữa bộ não và tay chân:

```
FLANNEL_NETWORK=10.244.0.0/16  
FLANNEL_SUBNET=10.244.1.1/24  
FLANNEL_MTU=1450  
FLANNEL_IPMASQ=true
```

FDB và ARP: "Bản đồ" định tuyến 3 bước của VTEP

Trên `worker1`, khi gửi packet đến Pod B (`10.244.2.7` trên `worker2`):

```
1. Route lookup:
   Đích 10.244.2.0/24 via 10.244.2.0 dev flannel.1 onlink (next hop là 10.244.2.0)
      ↓
2. ARP table (flannel.1):
   10.244.2.0 → MAC cc:dd:ee:ff:11:22 (VTEP MAC của worker2)
      ↓
3. FDB table (flannel.1):
   cc:dd:ee:ff:11:22 → dst 192.168.64.12 (IP vật lý của worker2)
      ↓
   Tạo VXLAN UDP packet (Dst Port: 8472) → gửi đến 192.168.64.12
```

- **Route:** Subnet IP Pod đích đi qua VTEP nào?
- **ARP:** VTEP gateway đối diện có MAC là gì?
- **FDB:** MAC VTEP đó tương ứng với IP vật lý nào của Node?

Lab Time: Dựng Flannel & Mổ xẻ cơ chế định tuyến

Chúng ta sẽ thực hành các bước sau trong file hướng dẫn `lab-guide.md`:

1. **Quan sát cụm trắng:** Xem xét trạng thái `NotReady` của node và sự thiếu hụt định tuyến.
2. **Cài đặt & Theo dõi:** Dựng Flannel CNI và quan sát card ảo `flannel.1`, bridge `cni0` xuất hiện.
3. **Trace Route, ARP, FDB:** Soi tận mắt 3 bảng dữ liệu mà Kernel Linux dùng để điều hướng packet.
4. **Giả lập sự cố:** Tự tay tái hiện lỗi sai card mạng chính (`--iface`), mất cấu hình CNI (kết `Terminating`), tường lửa chặn VXLAN (UDP 8472), và cạn kiệt IPAM lease.

👉 Hãy làm theo các bước chi tiết trong file `lab-guide.md`

Key Takeaways

Phân công trách nhiệm:

- **K8s API:** Lưu trữ `podCIDR` allocation và Node annotations (MAC VTEP, public-ip).
- **flanneld:** Watch API, setup `flannel.1` interface, điền các bảng ARP/FDB/Route, ghi `subnet.env`.
- **CNI plugin:** Đọc `subnet.env`, delegate xuống bridge plugin để cắm mạng và gán IP cho Pod.

Bảng tra cứu tĩnh:

Nhờ `flanneld` đồng bộ tĩnh các bảng ARP và FDB từ trước, Kernel có thể đóng gói VXLAN và gửi gói tin cross-node ngay lập tức mà không cần bất kỳ giao thức khám phá động (dynamic discovery) nào khi packet truyền qua.

Tập tiếp theo: *VXLAN Backend — tcpdump soi gói tin thực tế, 50 bytes overhead đến từ đâu?*